

Document Number: P0194R1, ISO/IEC JTC1 SC22 WG21
Date: 2016-05-17
Project: Programming Language C++
Audience: Reflection(SG7) / EWG
Authors: Matúš Chochlík (chochlik@gmail.com)
Axel Naumann (axel.naumann@cern.ch)

Static reflection

Table of Contents

- [Introduction](#)
- [Revision history](#)
- [Proposed wording](#)
- [Acknowledgments](#)

Introduction

This paper is the fifth revision of the proposal to add support for compile-time reflection to standard C++. We propose that the compiler shall generate *metaobjects* — representations of certain program declarations, which can be reasoned about at compile time. These metaobjects can then be used, through a set of *operations* to obtain various pieces of metadata, like declaration names, lists of scope members, information about specifiers, and so on. The *metaobjects* are implemented as anonymous types conforming to defined concepts, and the operations are implemented as class templates.

This paper is accompanied by another paper — P0385R0, which discusses the use cases, rationale, design decisions, future evolution of the proposed reflection facility and contains multiple examples of use and replies to frequently asked questions.

Revision history

Revision 1 ([N3996](#))

Describes the method of static reflection by the means of compiler-generated anonymous types. Introduces the first version of the metaobject concepts and some possibilities of their implementation. Also includes discussion about the motivation and the design rationale for the proposal.

Revision 2 ([N4111](#))

Refines the metaobject concepts and introduces a concrete implementation of their interface by the means of templates similar to the standard type traits. Describes some additions to the standard library (mostly meta-programming utilities), which simplify the use of the metaobjects. Answers some questions from the discussion about N3996 and expands the design rationale.

Revision 3 ([N4451](#))

Incorporates the feedback from the discussion about N4111 at the Urbana meeting, most notably reduces the set of metaobject concepts and refines their definitions, removes some of the additions to the standard library added in the previous revisions. Adds context-dependent reflection.

Revision 4 ([P0194R0](#))

Further refines the concepts from N4111; prefixes the names of the metaobject operations with `get_`, adds new operations, replaces the metaobject category tags with new metaobject traits. Introduces a nested namespace `std::meta` which contains most of the reflection-related additions to the standard library. Rephrases definition of meta objects using Concepts Lite. Specifies the reflection operator name — `reflexpr`. Introduces an experimental implementation of the reflection operator in clang. Drops the context-dependent reflection from N4111 (will be re-introduced later).

Revision 5 (P0194R1)

Dropped all metaobject traits except `is_metaobject`. All metaobject classification is now done by using the concepts. We assume that the following syntax of the Concepts specification is implemented:

```
template <T>
constexpr bool Concept = unspecified;
```

and also assuming that the Concepts TS Issue 29 is resolved and that `Concept<T>` is a generally usable boolean expression, so that the following is valid code:

```
static_assert(Concept<T>, " ... ");
using U = std::conditional_t<Concept<T>, T1, T2>;
if(Concept<T>) { /* ... */ }
```

The `meta::Scoped` concept has been renamed to `meta::ScopeMember`. The `meta::Constant` and `meta::Specifier` concepts, and several new operations have been added.

The aliases for the operation templates returning metaobjects had previously the `_t` suffix; this has been changed to the `_m` suffix. For example:

```
get_type_t -> get_type_m
get_scope_t -> get_scope_m
get_aliased_t -> get_aliased_m
get_data_members_t -> get_data_members_m
```

Proposed wording

Insert a new element in Table 1, C++ *library headers* of [general.namespaces], named `<experimental/reflexpr>`.

Insert a new section:

? Compile-time reflection	[refl]
?.1 General description	[refl.general]
Compile-time constant metadata describing various aspects of a compiled program are provided indirectly by the means of types generated by the compiler — <i>Metaobjects</i>. A <i>Metaobject</i> is an anonymous type <i>reflecting</i> (representing) a particular declaration in a C++ program, the base-level program declaration. It can be reasoned-about at compile-time and provides access to <i>metadata</i> describing various properties of the reflected declaration through a set of templates. The <i>Metaobject</i> types themselves are opaque, without any visible internal structure. Values of such a <i>Metaobject</i> type cannot be created. The <i>Metaobjects</i> are generated on demand by the compiler as the result of the invocation of the <code>reflexpr</code> operator. The actual metadata can be obtained by instantiating templates constituting the <i>interface</i> of the <i>Metaobjects</i>. Each template serves a particular purpose and provides a single piece of metadata. These templates are collectively referred to as <i>Metaobject operations</i>. <i>Metaobjects</i> reflecting different kinds of declarations conform to different <i>Metaobject concepts</i> and have different interfaces — sets of templates applicable to the <i>Metaobjects</i>. These concepts can also be used for <i>Metaobject</i> classification. <i>Metaobject</i> concepts form a generalization-specialization hierarchy, with <code>meta::Object</code> being the common generalization for all <i>Metaobjects</i>.	
?.2 Metaobject trait	[refl.trait]
A new <i>UnaryTypeTrait</i> — <code>is_metaobject</code> is added to section [meta.unary.cat] to distinguish between <i>Metaobject</i> types generated by the compiler and all other types. <pre>namespace std { namespace experimental { template <typename T> struct is_metaobject; template <typename T> constexpr bool is_metaobject_v = is_metaobject<T>::value;</pre>	

```

} // namespace experimental
} // namespace std

```

[Example:

```

is_metaobject_v<int>; // false
is_metaobject_v<void>; // false
is_metaobject_v<string>; // false
is_metaobject_v<reflexpr(int)>; // true
is_metaobject_v<reflexpr(std)>; // true
is_metaobject_v<reflexpr(std::string)>; // true

```

— end example]

?3 Header <experimental/reflexpr> synopsis

[refl.synopsis]

```

namespace std {
namespace experimental
namespace meta {

// metaobject concepts
template <typename T> concept bool Object;
template <Object T> concept bool ObjectSequence;
template <Object T> concept bool Reversible;
template <Object T> concept bool Named;
template <Object T> concept bool Specifier;
template <Object T> concept bool Typed;
template <Object T> concept bool ScopeMember;
template <Object T> concept bool Scope;
template <Object T> concept bool Alias;
template <Object T> concept bool ClassMember;
template <Object T> concept bool Linkable;
template <Object T> concept bool Namespace;
template <Object T> concept bool GlobalScope;
template <Object T> concept bool NamespaceAlias;
template <Object T> concept bool Type;
template <Object T> concept bool TypeAlias;
template <Object T> concept bool Class;
template <Object T> concept bool Enum;
template <Object T> concept bool EnumClass;
template <Object T> concept bool Constant;
template <Object T> concept bool Variable;
template <Object T> concept bool DataMember;
template <Object T> concept bool MemberType;
template <Object T> concept bool EnumValue;

// metaobject operations
template <Object T1, Object T2> struct reflects_same;
template <Object T> struct get_source_location;
template <ObjectSequence T> struct get_size;
template <ObjectSequence T1, size_t Index> struct get_element;
template <Named T> struct get_name;
template <Typed T> struct get_type;
template <ScopeMember T> struct get_scope;
template <Alias T> struct get_aliased;
template <ClassMember T> struct is_public;
template <ClassMember T> struct is_protected;
template <ClassMember T> struct is_private;
template <ClassMember T> struct get_access_specifier;
template <Linkable T> struct is_static;
template <Type T> struct get_reflected_type;
template <Class T> struct get_elaborated_type_specifier;
template <Class T> struct get_data_members;
template <Class T> struct get_all_data_members;
template <Class T> struct get_member_types;
template <Class T> struct get_all_member_types;
template <Enum T> struct get_enum_values;
template <Constant T> struct get_constant;
template <Variable T> struct get_pointer;

//
template <Object T1, Object T2>
constexpr auto reflects_same_v = reflects_same<T1, T2>::value;
template <ObjectSequence T>
constexpr auto get_size_v = get_size<T>::value;
template <ObjectSequence T1, size_t Index>
using get_element_m = typename get_element<T1, Index>::type;
template <Named T>
constexpr auto get_name_v = get_name<T>::value;
template <Typed T>
using get_type_m = typename get_type<T>::type;
template <ScopeMember T>
using get_scope_m = typename get_scope<T>::type;
template <Alias T>
using get_aliased_m = typename get_aliased<T>::type;
template <ClassMember T>
constexpr auto is_public_v = is_public<T>::value;
template <ClassMember T>
constexpr auto is_protected_v = is_protected<T>::value;
template <ClassMember T>
constexpr auto is_private_v = is_private<T>::value;
template <ClassMember T>
using get_access_specifier_m = typename get_access_specifier<T>::type;
template <Linkable T>
constexpr auto is_static_v = is_static<T>::value;

```

```

template <Type T>
    using get_reflected_type_t = typename get_reflected_type<T>::type;
template <Class T>
    using get_elaborated_type_specifier_m = typename get_elaborated_type_specifier<T>::type;
template <Class T>
    using get_data_members_m = typename get_data_members<T>::type;
template <Class T>
    using get_all_data_members_m = typename get_all_data_members<T>::type;
template <Class T>
    using get_member_types_m = typename get_member_types<T>::type;
template <Class T>
    using get_all_member_types_m = typename get_all_member_types<T>::type;
template <Enum T>
    using get_enum_values_m = typename get_enum_values<T>::type;
template <Constant T>
    constexpr auto get_constant_v = get_constant<T>::value;
template <Variable T>
    const auto get_pointer_v = get_pointer<T>::value;

} // namespace meta
} // namespace experimental
} // namespace std

```

?.4 *Metaobject* concepts

[[refl.concepts](#)]

Concept	Requirements	Optional restrictions	Description
template <typename T> concept bool Object;	is_metaobject_v<T>		meta::Object is a concept modelled by stateless anonymous types generated by the compiler which allow access to metadata reflecting specific program declarations.
template <typename T> concept bool ObjectSequence;	Object<T>		meta::ObjectSequence is an ordered sequence of metaobjects.
template <typename T> concept bool Reversible;	Object<T>		meta::Reversible is a metaobject that can be reverted to the original declaration.
template <typename T> concept bool Named;	Object<T>		meta::Named reflects a named declaration.
template <typename T> concept bool Specifier;	Named<T>		meta::Specifier reflects a specifier (const, virtual, static, noexcept, public, etc.).
template <typename T> concept bool Typed;	Object<T>		meta::Typed reflects a base-level declaration with a type (for example a variable).
template <typename T> concept bool ScopeMember;	Object<T>		meta::ScopeMember reflects a declaration nested in a scope (like the global scope, namespace, class, strongly-typed enums, etc.).
template <typename T> concept bool Scope;	Object<T>		meta::Scope reflects a scope.
template <typename T> concept bool Alias;	Named<T>		meta::Alias reflects a type or namespace alias.
template <typename T> concept bool ClassMember;	ScopeMember<T> Class<get_scope_m<T>>		meta::ClassMember reflects a class member.
template <typename T> concept bool Linkable;	Named<T> ScopeMember<T>		meta::Linkable reflects a declaration with storage duration and/or linkage.
template <typename T>	Scope<T>	Named<T> ScopeMember<T>	meta::Namespace reflects a

concept bool Namespace;			namespace.
template <typename T> concept bool GlobalScope;	Namespace<T> not(Named<T>) not(ScopeMember<T>)		meta::GlobalScope reflects the global scope.
template <typename T> concept bool NamespaceAlias;	Namespace<T> Alias<T> Namespace<get_aliased_m<T>>		meta::NamespaceAlias reflects a namespace alias.
template <typename T> concept bool Type;	ScopeMember<T> Reversible<T>	Named<T>	meta::Type reflects a type.
template <typename T> concept bool TypeAlias;	Type<T> Alias<T> Type<get_aliased_m<T>>		meta::TypeAlias reflects a type alias or typedef.
template <typename T> concept bool Class;	Type<T> Scope<T>		meta::Class reflects a class.
template <typename T> concept bool Enum;	Type<T>		meta::Enum reflects an enumeration.
template <typename T> concept bool EnumClass;	Enum<T> Scope<T>		meta::EnumClass reflects a strongly typed enumeration.
template <typename T> concept bool Constant;	Named<T> Typed<T>		meta::Constant reflects a compile-time constant value.
template <typename T> concept bool Variable;	Named<T> Typed<T> Linkable<T>		meta::Variable reflects a variable.
template <typename T> concept bool DataMember;	Variable<T> ClassMember<T>		meta::DataMember reflects a class data member.
template <typename T> concept bool MemberType;	Type<T> ClassMember<T>		meta::MemberType reflects a class member type (typedef, class, union, enum).
template <typename T> concept bool EnumValue;	Constant<T> ScopeMember<T> Enum<get_type_m<T>>		meta::EnumValue reflects an enum value.

[*Note: Metaobjects* reflecting anonymous types, anonymous namespaces or the global scope do not satisfy the meta::Named concept. This means that they do not implement the meta::get_name operation. See below. — *end note*]

[*Example:*

```
static_assert(meta::Named<reflexpr(int)>, "");
static_assert(meta::Named<reflexpr(string)>, "");

auto l = [](void) { };
static_assert(!meta::Named<reflexpr()>, "");
static_assert(!meta::Named<reflexpr(:)>, "");
static_assert(!meta::Named<reflexprdecltype(l)>, "");
```

— *end example*]

[*Note: Metaobjects* reflecting the global scope do not satisfy the meta::ScopeMember concept. This means that they do not implement the meta::get_scope operation. See below. — *end note*]

[*Example:*

```
static_assert(meta::ScopeMember<reflexpr(int)>, "");
static_assert(meta::ScopeMember<reflexpr(string)>, "");

static_assert(!meta::ScopeMember<reflexpr(:)>, "");
static_assert(!meta::ScopeMember<reflexprdecltype(l)>, "");
```

— *end example*]

[*Note: Metaobjects* reflecting strongly-typed enumerations conform to the meta::Scope concept, regular enumerations do not. — *end note*]

[*Example:*

```
enum weak_enum { a, b, c };
enum class scoped_enum { d, e, f };

using meta_we = reflexpr(weak_enum);
using meta_se = reflexpr(scoped_enum);

static_assert(meta::Enum<meta_we>, "");
```

```
static_assert(meta::Enum<meta_se>, "");
static_assert(!meta::Scope<meta_we>, "");
static_assert( meta::Scope<meta_se>, "");
```

— *end example*]

?5.5 *Metaobject* operations

[refl.ops]

A *MetaobjectOperation* returns a single piece of metadata describing a particular aspect of the declaration reflected by a *Metaobject*. It is a class template taking one or more arguments, at least one of which is a model of the *Metaobject* concept. The result of a *MetaobjectOperation* can be either a compile-time constant value a `const` pointer or a type.

?5.5.1 Boolean constant result

[refl.ops.bool]

All *MetaobjectOperation* templates returning a boolean value derive publicly and unambiguously from `true_type` OR `false_type`.

For example:

```
struct MetaobjectOperation
: integral_constant<bool, implementation-defined>
{ };
```

?5.5.2 Integral or `enum` constant result

[refl.ops.int]

All *MetaobjectOperation* templates returning an integer or `enum` value derive publicly and unambiguously from a specialization of `integral_constant` with appropriate integer or `enum` type as the first argument.

For example:

```
struct MetaobjectOperation
: integral_constant<integral-or-enum-type, implementation-defined>
{ };
```

?5.5.3 String constant result

[refl.ops.str]

All *MetaobjectOperation* templates returning a character string instantiate into a type *equivalent* to the following:

```
struct MetaobjectOperation
{
    typedef const char value_type[N+1];
    static constexpr value_type value;
};
```

where `N` is the length of the returned string and the last element in the `value` character array is the `'\0'` character. The member names `value_type`, `value`, type are publicly and unambiguously available in such *MetaobjectOperation*.

?5.5.4 *Metaobject* result

[refl.ops.metaobj]

The *MetaobjectOperation* templates yielding another *Metaobject* instantiate into a type *equivalent* to the following:

```
struct MetaobjectOperation
{
    typedef meta::Object type;
};
```

The member name `type` is publicly and unambiguously available in such *MetaobjectOperation*.

?5.5.5 Type result

[refl.ops.type]

The *MetaobjectOperation* templates yielding a non-*Metaobject* type instantiates into a type *equivalent* to the following:

```
struct MetaobjectOperation
{
    typedef unspecified type;
};
```

The member name type is publicly and unambiguously available in a *MetaobjectOperation*.

?5.6 Pointer result

[refl.ops.ptr]

The *MetaobjectOperation* templates returning a pointer or a class data member pointer instantiate into a type *equivalent* to the following:

```
struct MetaobjectOperation
{
    typedef conditional_t<
        bool(is-non-static-class-data-member),
        unspecified-type unspecified-class::*,
        unspecified-type*
    > value_type;

    static const value_type value;
};
```

?5.7 Operation description

[refl.ops.desc]

Template	Description	Result
template <Object T1, Object T2> struct reflects_same;	indicates if two metaobjects reflect the same base-level declaration.	a boolean constant
template <Object T> struct get_source_location;	returns the source location info of the declaration of a base-level program declaration reflected by a <i>meta::Object</i> .	a <i>std::source_location</i> constant
template <ObjectSequence T> struct get_size;	returns the number of elements in the sequence.	a <i>size_t</i> integral constant
template <ObjectSequence T1, size_t Index> struct get_element;	returns the i-th <i>meta::Object</i> in a <i>meta::ObjectSequence</i> .	<i>meta::Object</i>
template <Named T> struct get_name;	returns the basic name of the a named declaration reflected by a <i>meta::Named</i> .	a string constant
template <Typed T> struct get_type;	returns the <i>meta::Type</i> reflecting the type of a base-level declaration with a type (for example a variable) reflected by a <i>meta::Typed</i> .	<i>meta::Type</i>
template <ScopeMember T> struct get_scope;	returns the <i>meta::Scope</i> reflecting the scope of a declaration nested in a scope (like the global scope, namespace, class, strongly-typed enums, etc.) reflected by a <i>meta::ScopeMember</i> .	<i>meta::Scope</i>
template <Alias T> struct get_aliased;	returns the <i>meta::Named</i> reflecting the original declaration of a type or namespace alias reflected by a <i>meta::Alias</i> .	<i>meta::Named</i>
template <ClassMember T> struct is_public;	returns whether the a class member reflected by a <i>meta::ClassMember</i> was declared with public access.	a boolean constant
template <ClassMember T> struct is_protected;	returns whether the a class member reflected by a <i>meta::ClassMember</i> was declared with protected access.	a boolean constant
template <ClassMember T> struct is_private;	returns whether the a class member reflected by a <i>meta::ClassMember</i> was declared with private access.	a boolean constant
template <ClassMember T> struct get_access_specifier;	returns the <i>meta::Specifier</i> reflecting the access specifier of a class member reflected by a <i>meta::ClassMember</i> .	<i>meta::Specifier</i>
template <Linkable T> struct is_static;	returns whether the a declaration with storage duration and/or linkage reflected by a <i>meta::Linkable</i> was	a boolean constant

	declared with the static specifier.	
template <Type T> struct get_reflected_type;	returns the the base-level type reflected by a meta::Type.	the original type reflected by the argument
template <Class T> struct get_elaborated_type_specifier;	returns a meta::Specifier reflecting the elaborated type specifier (class,struct,union,...) of a class reflected by a meta::Class.	meta::Specifier
template <Class T> struct get_data_members;	returns a meta::ObjectSequence of meta::DataMember(s) reflecting the public data members of a class reflected by a meta::Class.	meta::ObjectSequence of meta::DataMember(s)
template <Class T> struct get_all_data_members;	returns a meta::ObjectSequence of meta::DataMember(s) reflecting all (including the private and protected) data members of a class reflected by a meta::Class.	meta::ObjectSequence of meta::DataMember(s)
template <Class T> struct get_member_types;	returns a meta::ObjectSequence of meta::MemberType(s) reflecting the public member types of a class reflected by a meta::Class.	meta::ObjectSequence of meta::MemberType(s)
template <Class T> struct get_all_member_types;	returns a meta::ObjectSequence of meta::DataMember(s) reflecting all (including the private and protected) member types of a class reflected by a meta::Class.	meta::ObjectSequence of meta::DataMember(s)
template <Enum T> struct get_enum_values;	returns a meta::ObjectSequence of meta::Constant(s) reflecting all enum values of an enumeration reflected by a meta::Enum.	meta::ObjectSequence of meta::Constant(s)
template <Constant T> struct get_constant;	returns the value of a compile-time constant value reflected by a meta::Constant.	a constant value of an unspecified, integer or enum type
template <Variable T> struct get_pointer;	returns a pointer to the a variable reflected by a meta::Variable. If the variable is a class member then the pointer is a class data member pointer, otherwise it is a plain pointer.	a regular or class data member pointer

For meta::Types reflecting native types with modifiers like unsigned, long, etc. the get_name operation returns the full type specifier name.

[Example:

```
meta::get_name_v<reflexpr(unsigned)>      // "unsigned int"
meta::get_name_v<reflexpr(short)>         // "signed short int"
meta::get_name_v<reflexpr(long long)>     // "signed long long int"
meta::get_name_v<reflexpr(signed char)>    // "signed char"
meta::get_name_v<reflexpr(unsigned char)> // "unsigned char"
meta::get_name_v<reflexpr(char)>          // "char"
```

— end example]

The get_name invoked on a meta::Alias returns the alias name not the name of the aliased declaration.

[Example:

```
using foo = int;
using bar = foo;
namespace std = baz;

meta::get_name_v<reflexpr(int)>      // "int"
meta::get_name_v<reflexpr(std)>     // "std"
meta::get_name_v<reflexpr(foo)>     // "foo"
meta::get_name_v<reflexpr(bar)>     // "bar"
meta::get_name_v<reflexpr(baz)>     // "baz"
```

— end example]

The `get_name` operation returns the type, namespace, etc. name without any qualifiers, asterisks, ampersands, angle or square brackets, etc.

[Example:

```
using meta::get_name_v;

get_name_v<reflexpr(std::vector<int>>> // "vector"
get_name_v<reflexpr(volatile std::size_t* [10])> // "size_t"
get_name_v<reflexpr(std::set<int>* (*)(std::vector<double>&))> // ""
get_name_v<reflexpr(std::set<int>* (*pfoo)(std::set<double>&))> // "pfoo"
get_name_v<reflexpr(std::set<int>* foo(std::vector<double>&))> // "foo"
get_name_v<reflexpr(std::chrono)> // "chrono"
get_name_v<reflexpr(std::launch)> // "launch"
get_name_v<reflexpr(static)> // "static"
get_name_v<reflexpr(public)> // "public"
get_name_v<reflexpr(class)> // "class"
```

— end example]

Template type parameters are also considered to be type aliases:

[Example:

```
template <typename A, typename B>
void foo(A a, B b)
{
    using meta::get_name_v;
    using meta::get_aliased_m;

    using mA = reflexpr(A);
    using mB = reflexpr(B);

    cout << "1:" << get_name_v<mA> << endl;
    cout << "2:" << get_name_v<get_aliased_m<mA>> << endl;
    cout << "3:" << get_name_v<get_aliased_m<get_aliased_m<mA>>> << endl;

    cout << "4:" << get_name_v<mB> << endl;
    cout << "5:" << get_name_v<get_aliased_m<mB>> << endl;
}

using rank_t = int;
rank_t a = 123;
float b = 45.67;

foo(a, b);
```

produces the following output:

```
1:A
2:rank_t
3:int
4:B
5:float
```

— end example]

The `meta::get_name` operation cannot be invoked on *Metaobjects* which do not conform to the `meta::Named` concept.

[Example:

```
auto l = [](void) { };
meta::get_name_v<reflexpr(::)> // ill-formed
meta::get_name_v<reflexpr(decltype(l))> // ill-formed
```

— end example]

[Note: The *Metaobjects* in a `meta::ObjectSequence` are ordered according to the order of their declarations in the translation unit. — end note]

[Example:

```
struct foo
{
    typedef int t1;
    typedef float t2;

    t1 a, b, c;
    t2 d;
};

using meta_foo = reflexpr(foo);

using meta::get_element_m;
using meta::get_data_members_m;
using meta::get_member_types_m;

get_name_v<get_element_v<get_member_types_m<meta_foo>, 0>> // "t1"
get_name_v<get_element_v<get_member_types_m<meta_foo>, 1>> // "t2"

get_name_v<get_element_v<get_data_members_m<meta_foo>, 0>> // "a"
get_name_v<get_element_v<get_data_members_m<meta_foo>, 1>> // "b"
```

```
get_name_v<get_element_v<get_data_members_m<meta_foo>, 2>> // "c"
get_name_v<get_element_v<get_data_members_m<meta_foo>, 3>> // "d"
```

— *end example*

[*Note:* The `meta::ObjectSequence` returned by `meta::get_data_members`, `meta::get_all_data_members`, `meta::get_member_types` and `meta::get_all_member_types` do not include *Metaobjects* reflecting inherited class members. — *end note*]

[*Example:*

```
struct foo
{
    typedef int attr_t;
    attr_t a, b, c, d;
};

struct bar : foo
{
    float e, f;
};

using meta_foo = reflexpr(foo);
using meta_bar = reflexpr(bar);

using meta::get_size_v;
using meta::get_data_members_m;
using meta::get_member_types_m;

get_size_v<get_data_members_m<meta_foo>> // 4
get_size_v<get_data_members_m<meta_bar>> // 2

get_size_v<get_member_types_m<meta_foo>> // 1
get_size_v<get_member_types_m<meta_bar>> // 0
```

— *end example*

[*Note:* The `meta::ObjectSequence` returned by `meta::get_data_members` and `meta::get_member_types` contain only the *Metaobjects* reflecting publicly accessible class members. The `meta::ObjectSequence` returned by `meta::get_all_data_members` and `meta::get_all_member_types` contain *Metaobjects* reflecting all class members including those which are not publicly accessible otherwise. — *end note*]

[*Example:*

```
struct foo
{
private:
    typedef int _attr_t;
    _attr_t _a, _b, _c;
public:
    typedef float value_type;
    typedef size_t size_type;
    value_type d;
};

using meta_foo = reflexpr(foo);

using meta::get_size_v;
using meta::get_name_v;
using meta::get_element_m;
using meta::get_data_members_m;
using meta::get_all_data_members_m;
using meta::get_member_types_m;
using meta::get_all_member_types_m;

get_size_v<get_data_members_m<meta_foo>> // 1
get_size_v<get_all_data_members_m<meta_foo>> // 4

get_size_v<get_member_types_m<meta_foo>> // 2
get_size_v<get_all_member_types_m<meta_foo>> // 3

get_name_v<get_element_m<get_data_members_m<meta_foo>, 0>> // "d"
get_name_v<get_element_m<get_all_data_members_m<meta_foo>, 0>> // "_a"
```

— *end example*

[*Note:* There is a dual interface for retrieving specifier-related metadata. One is using the `meta::Specifier` concept and its operations which is more generic, and another using the boolean operations like `meta::is_static`, `meta::is_public`, etc. which may be more convenient in simpler use-cases. — *end note*]

[*Example:*

```
struct S
{
private:
    int i;
public:
    float f;
};

using meta_S = reflexpr(S);
```

```

using meta_S_i = meta::get_element_m<meta::get_data_members_m<meta_S>, 0>;
using meta_S_f = meta::get_element_m<meta::get_data_members_m<meta_S>, 1>;

using meta::reflects_same_v;
using meta::get_access_specifier_m;

reflects_same_v<reflexpr(private), get_access_specifier_m<meta_S_i>> // true
reflects_same_v<reflexpr(public), get_access_specifier_m<meta_S_i>> // false
reflects_same_v<reflexpr(public), get_access_specifier_m<meta_S_f>> // true

meta::is_private<meta_S_i> // true
meta::is_public<meta_S_i> // false
meta::is_public<meta_S_f> // true

```

— *end example*]

?5.8 Utility operations

[refl.ops.util]

The `unpack_sequence` operation applicable to models of `meta::ObjectSequence` is added to the `<reflexpr>` header and is *equivalent* to:

```

namespace std {
namespace experimental {
namespace meta {

template <ObjectSequence Seq, template <typename ...> class Tpl>
struct unpack_sequence
{
    typedef Tpl<
        get_element_m<Seq, 0>,
        get_element_m<Seq, 1>,
        ...,
        get_element_m<Seq, get_size_v<Seq>-1>,
    > type;
};

template <ObjectSequence Seq, template <typename ...> class Tpl>
using unpack_sequence_t = typename unpack_sequence<Seq, Tpl>::type;

} // namespace meta
} // namespace experimental
} // namespace std

```

[*Example*:

```

struct foo
{
    int a,
    bool b;
    char c;
    double d;
}

template <meta::DataMember ... MDM>
using helper = tuple<meta::get_reflected_type_t<MDM>...>;

using X = meta::unpack_sequence_t<
    meta::get_data_members<reflexpr(foo)>,
    helper
>;

is_same_v<X, tuple<int, bool, char, double>> // true

```

— *end example*]

?6 The reflection operator

[refl.reflexpr]

The value of the `reflexpr` is a type satisfying `meta::Object` and other `meta` concepts, depending on the operand. The returned type is implementation defined. Meta-operations on it describe the operand.

The `reflexpr` operator takes an *id-expression* (5.1), or *access-specifier* (10) as operand, or nothing. The *id-expression* must be uniquely resolved through regular lookup, as if used outside of the `reflexpr` operator. Otherwise, the expression is ill-formed.

- If the operand is a *typedef-name*, the result will be satisfying `TypeAlias`.
- Otherwise, if the operand is a *class-name*, the result will be satisfying `Class`.
- Otherwise, if the operand is a *enum-name*, the result will be satisfying `EnumClass` if the *enum-name* refers to a *scoped enumerator* (7.2), or `Enum` if the *enum-name* refers to an *unscoped enumerator*.
- Otherwise, if the operand is a *type-name*, the result will be satisfying `Type`.
- Otherwise, if the operand is the name of a *namespace-alias-declaration*, the result will be satisfying `NamespaceAlias`.
- Otherwise, if the operand is a *namespace-name*, the result will be satisfying `Namespace`.

If the *id-expression* is a *type-name*, and the scope of its declaration is a class, then the result of `reflexpr` is

also satisfying `MemberType`.

The invocation of `reflexpr` on all other kinds of *id-expressions* is ill-formed.

[Example:

```
reflexpr() // reflects the global namespace
reflexpr::() // reflects the global namespace
reflexpr(std) // reflects the namespace
reflexpr(std::size_t) // reflects the type
reflexpr(std::launch) // reflects the enum type
reflexpr(std::vector<int>) // reflects the class
```

```
reflexpr(1) // ill-formed
reflexpr(std::sin) // ill-formed
reflexpr(std::vector) // ill-formed
reflexpr(is_same_v<void,void>) // ill-formed
```

— end example]

If the operand is an *access-specifier*, then the result of `reflexpr` is satisfying `Specifier`.

?6.1 Redclarations

[refl.reflexpr.redecl]

The meta data queried by `reflexpr` depends on the point of invocation of the operator; only declarations appearing in the translation unit before the invocation of the `reflexpr` operator will be visible. Subsequent invocations are independent of prior invocations, as if all compiler generated types were unique to each `reflexpr` invocation.

[Example:

```
using meta::get_size_v;
using meta::get_data_members_m;

struct foo;
using meta_foo_fwd1 = reflexpr(foo);
constexpr size_t n1 = get_size_v<get_data_members_m<meta_foo_fwd1>>; // 0

struct foo;
using meta_foo_fwd2 = reflexpr(foo);
constexpr size_t n2 = get_size_v<get_data_members_m<meta_foo_fwd2>>; // 0

using meta::reflects_same_v;

constexpr bool b1 = is_same_v<meta_foo_fwd1, meta_foo_fwd2>; // unspecified
constexpr bool b2 = reflects_same_v<meta_foo_fwd1, meta_foo_fwd2>; // true

struct foo { int a,b,c,d; };
using meta_foo = reflexpr(foo);
constexpr size_t n3 = get_size_v<get_data_members_m<meta_foo>>; // 4

constexpr bool b3 = is_same_v<meta_foo_fwd1, meta_foo>; // false
constexpr bool b4 = reflects_same_v<meta_foo_fwd1, meta_foo>; // true
```

— end example]

?6.2 Required header

[refl.reflexpr.header]

If the header `<reflexpr>` is not included prior to a use of `reflexpr`, the program is ill-formed. This header file makes available the definitions of the concepts and the templates implementing the operations on *Metaobjects* as described above.

?6.3 Reverse reflection

[refl.reflexpr.rev]

If a *Metaobject* conforming to the `meta::Reversible` concept is passed as the argument to the `reflexpr` operator, then this invocation has the same effect as if the name of the original declaration reflected by the *Metaobject* was used.

For instance if a `meta::Type` is passed as the argument, then `reflexpr` yields the reflected type. In this case the result of reverse reflection is the same as the result of the `meta::get_reflected_type` operation.

[Example:

```
using meta_int = reflexpr(int);
is_same_t<int, reflexpr(meta_int)>; // true
is_same_t<int, meta::get_reflected_type_t<meta_int>>; // true

reflexpr(meta_int) x = 123; // valid (same as int x;)

struct my_struct
{
    reflexpr(meta_int) i; // valid (same as int i;)
};
```

— *end example*]

Passing a *Metaobject* to the `reflexpr` operator is ill-formed, unless the *Metaobject* conforms to the `meta::Reversible` concept. — *end note*]

Acknowledgments

Thanks to Ricardo Fabiano de Andrade, Roland Bock and Klaim - Joël Lamotte who provided very valuable feedback, criticism and suggestions.

References

- [P0194R0] Chochlík M., Naumann A. — P0194R0 - Static reflection (revision 4)
- [N4451] Chochlík M. — N4451 - Static reflection (revision 3)
- [N4111] Chochlík M. — N4111 - Static reflection (revision 2)
- [N3996] Chochlík M. — N3996 - Static reflection